SP-33 Green: Mindful Micro-Journaling App

Final Report

CS4850 - Section 01 - Spring 2026

David Cardenas, David Berrospi, Siri Sriramoju, Jamie Coreas
Sharon Perry

April 21, 2026

Website link:
https://green-mindful-team.github.io/Mindful_WebPage/

Github link:
https://github.com/Green-Mindful-Team/Mindful_Moments_App.git

STATS and STATUS –

Total Man Hours – 234 hours

Status – Project is 100% complete

SP-33 Green: Mindful Micro-Journaling App 1
1.0 Introduction 4
2.0 Requirements 4
2.1 Design Constraints 4
2.1.0 Environment 4
2.1.1 User Characteristics 4
2.1.2 System 5
2.2 Functional Requirements 5
2.2.0 User Account Data 5
2.2.1 Journaling 5
2.2.2 Mood Tracking 5
2.2.3 Prompt Generation 5
2.2.4 Challenges (If Time Permits) 6
2.3 Non-Functional Requirements 6
2.3.0 Security 6
2.3.1 Capacity 6
2.3.2 Usability 6
2.4 External Interface Requirements 6
2.4.0 User Interface Requirements 6
2.4.1 Hardware Interface Requirements 7
2.4.2 Software Interface Requirements 7
2.4.3 Communication Interface Requirements 8
3.0 Analysis/Design 8
3.1 Design Considerations 8
3.1.0 Assumptions and Dependencies 8
3.1.1 General Constraints 8
3.1.2 Development Methods 9
3.2 Architectural Strategies 9
3.2.0 User Interface 10
3.3 System Architecture 10
3.4 Detailed System Design 11
3.4.0 Classification 11
3.4.1 Definition 12
3.4.2 Constraints 12
3.4.3 Resources 12
3.4.4 Interface/Exports 13
3.5 Technology Platform and Tools 13
4.0 Development 13
4.1 Implementation Detail Narrative 13
4.1.0 System Integration & Workflow Overview 13
Integrated Process: 14

Operational Workflow:....14
Global Design Decisions:....14
4.1.1 User Interface Component....15
4.1.2 Secure Local Data & Storage Layer....15
4.2 Database and External Connections....16
4.2.0 Connection Overview....16
4.2.1 Configuration Details....16
4.2.2 Integration Logic....17
4.3 Project Setup and Environment Configuration....17
4.3.0 Prerequisites:....17
4.3.1 Installation Steps:....17
4.3.2 Verification:....17
5.0 Test (plan and report)....18
5.1 Test Objectives....18
5.2 Scope of Testing....18
5.3 Test Cases....19
5.4 Test Procedures....20
5.5 Test Environment....21
5.6 Test Data....22
5.6.0 Mood Tracking Data....22
5.7 Software Test Report (STR)....22
6.0 Summary....23

1.0 Introduction

The mindful micro-journaling app is a smartphone application which is designed to encourage journaling and other mindful activities from its users. This goal is achieved by presenting the user with low effort prompts that can easily be answered in a few minutes and can be answered anywhere due to smartphones being easily portable. The design of the app places an emphasis on simplicity and security.

The primary features of the application include:

- Creation and editing of short journal entries
- Mood tracking associated with journal entries
- Viewing and browsing past journal entries
- AI-assisted writing prompts and affirmations
- Voice-based journaling through audio recording
- Local and secure storage of personal data

2.0 Requirements

2.1 Design Constraints

2.1.0 Environment

The application shall run on both android and iOS smartphones, rather than tablets or desktops. The application is designed to take advantage of the portability of these devices in order to allow users to journal whenever and wherever they like.
The application will be developed using **Expo**, which is a framework built on top of **React Native**, allowing the application to be deployed on both Android and iOS from a single codebase. The development environment will rely on modern web and mobile technologies and will support testing through emulators and physical devices. The application may optionally use internet connectivity for features such as AI-assisted prompts, but core functionality will be available without a constant internet connection.

2.1.1 User Characteristics

Users are expected to have basic knowledge of handling a smartphone or android phone. The target audience for this application is the general public, and minimal advanced knowledge shall be required to operate the application.

2.1.2 System

The system should have high performance on the majority of smartphones, it should load and respond to user input quickly. User data should be private and secured, journal entries will be stored locally. The system will be implemented using JavaScript and TypeScript through the React Native framework with Expo. The application will use standard mobile device functionality such as touch input, local storage, and optional audio recording for voice-based journaling. The system architecture focuses on modular functionality, allowing features such as journaling, mood tracking, and prompt selection to operate independently while sharing common data storage. The system will prioritize reliability, responsiveness, and data security.

2.2 Functional Requirements

2.2.0 User Account Data

- Users are allowed to use the application without creating an online account.
- Data such as journal entries will be stored locally on the user's device

2.2.1 Journaling

- The user should get a daily notification to prompt them to journal
- The app should present the user with a daily journaling prompt
- The user should be able to submit a short journal entry to every prompt
- Previous journal entries will be accessible to the user
- All journal entries should have a date and time associated with them
- The user should be allowed to create non-prompted journal entries if desired

2.2.2 Mood Tracking

- Users will indicate mood by selecting between icons representing various moods
- The user should be prompted to record their mood daily alongside daily journal entries
- The user should be allowed to record their mood even without submitting a journal entry
- Previous mood data should be accessible to the user
- All mood entries should have a date associated with them

2.2.3 Prompt Generation

- The system shall present journaling prompts selected from a predefined set of mindfulness prompts.
- Prompts may focus on themes such as reflection, gratitude, or emotional awareness.
- Users shall be able to view and use prompts while creating journal entries.

2.2.4 Challenges (If Time Permits)

- Users will be presented with “challenges” in addition to daily journaling
- Challenges will be optional and encourage users to participate in additional mindfulness activities
- When a user completes a challenge they will be rewarded with a badge

2.3 Non-Functional Requirements

2.3.0 Security

- App will protect user data and prevent unauthorized access
- User entries stored securely on device
- User data not shared with third parties

2.3.1 Capacity

- The application will support a large number of journal entries without performance suffering or degrading
- The app will support concurrent usage

2.3.2 Usability

- App will be easy to navigate for first time users
- Basic journaling functions will be readily available within a few number of interactions with the app UI
- App will provide clear and prompt feedback for user actions

2.4 External Interface Requirements

2.4.0 User Interface Requirements

Minimalist Aesthetic and Design

- Simple and uncluttered layout with clear labeling

- Clear design hierarchy within layout design that makes reading easy (ex., headers are largest font size in layout)
- Clear and consistent UI elements throughout app design (ex., Using the same font style for all headers)
- Visually harmonious color scheme that fits with the health and wellness theme of the app. (ex., neutral colors)

Navigation and Usability

- Intuitive and easily navigable UI
- Consistent placement of navigation elements
- Buttons and menus icons that are noticeable and make sense in terms of use
- Responsiveness throughout the app. Instant feedback is provided when user interacts with elements on the app (ex., user is notified when they enter and save a journal entry)

2.4.1 Hardware Interface Requirements

Touchscreen Input

- Touchscreen input will be supported. Expected user interaction includes: tapping, typing scrolling, and swiping.
- UI elements on app will be sized to function with touch screen input (ex., buttons that expect hand based interactions)

Display

- App will work with standard smartphone screen sizes and resolution
- Layout will scale proportionately based on screen dimensions and phone orientation

Storage

- App will work with the user's device storage capabilities to save app preferences/settings for user
- App will work with the user's device storage capabilities to save user journal entries
- If storage is insufficient for app use, users will be notified

Device Notifications

- The app will work with the user's device notification settings to deliver journaling reminders.

2.4.2 Software Interface Requirements

Mobile Operating System

- The app will work with mobile device operating systems (IOS and Android) to communicate, exchange data, and request services. Some standard functionalities include:
    - Background processes
    - Touchscreen input
    - System notifications

Local Data Management

- Secure data handling guaranteed by app to prevent unauthorized access to data
- User data stored/saved for all user sessions
- The app will utilize local file storage on user device to save user preferences and journal entries

2.4.3 Communication Interface Requirements

Network Connectivity

- The app will work over mobile and wifi network connections

Data Transference

- The app will transmit data while adhering to secure communication protocols
- Data between app and external services will be encrypted

Client-Server

- The app will use standardized APIs to communicate with backend services

3.0 Analysis/Design

3.1 Design Considerations

3.1.0 Assumptions and Dependencies

The system is designed under the assumption that it will be used on modern mobile devices with standard computing and storage capabilities. It is assumed that users will access the application through supported mobile operating systems and will have basic familiarity with common mobile interactions such as touch input and navigation. The system also assumes that users will primarily interact with the application individually and intermittently rather than in continuous sessions.

The design depends on the availability of basic device features such as local storage, audio input, and network connectivity for optional functionality. Some features may rely on external services, and it is assumed that these services may not always be available. The system is therefore designed to continue functioning with reduced features when such dependencies are unavailable. Future changes in functionality, such as enhanced analysis or data synchronization, are anticipated and have been considered in the overall design to allow for extension without major architectural changes.

3.1.1 General Constraints

Hardware and Software Constraints

- The app must be able to operate on a majority of smartphones
- Intensive tasks should be kept to a minimum to maximize performance
- Software failures should not lead to data loss

Network Constraints

- Functions requiring the internet should be kept to a minimum
- Tasks involving the internet should be optional for the user to participate in
- The user may not always have their device connected to the internet

User Data Constraints

- User data, such as journal entries, should be stored locally on the user's device
- Nobody, aside from the user, should be able to access any personal data that is stored locally on the User's device

3.1.2 Development Methods

The app will be developed using an iterative and incremental development approach. This approach was chosen after considering time constraints and the scope of the app. Development will see early testing and flexibility in design decisions as issues or requirements change.

Generally, an AGILE approach will be followed.

- Short development cycles
- Frequent evaluation of progress
- Continuous testing and integration of features
- Incremental delivery of features

3.2 Architectural Strategies

The proposed system is a mobile journaling application intended to support personal reflection, emotional awareness, and mental well-being. The overall architecture is designed to emphasize simplicity, privacy, and flexibility while remaining easy to extend in future iterations. Architectural decisions were guided by the need to support daily use, protect sensitive user data, and provide a calm and intuitive experience.

A cross-platform mobile approach was selected to allow the application to run on multiple mobile operating systems using a shared architectural design. This decision helps reduce development complexity and ensures consistent behavior across devices. Native-only development was considered but rejected due to higher development and maintenance effort,

while a purely web-based solution was deemed unsuitable because it would limit access to device features such as offline storage and audio input.

The system is designed around a local-first data strategy in which journal entries and mood data are stored directly on the user's device. This approach prioritizes privacy and ensures the application remains usable without an internet connection. Although cloud-based storage could allow synchronization across devices, it introduces additional security and dependency concerns and is therefore deferred to future versions.

Given the personal nature of journaling data, data protection is treated as a core architectural concern. The design includes mechanisms for encrypting stored information to prevent unauthorized access. While more advanced encryption techniques may be introduced later, the current design focuses on ensuring that sensitive content is not stored in an easily readable format.

The system also includes provisions for optional AI-assisted features such as writing prompts, affirmations, and reflective insights. These features are intended to enhance the user experience rather than define core functionality. The architecture is designed so that the system remains fully functional even when AI-based services are unavailable, ensuring reliability and independence from external services.

User interaction is guided by a minimalistic and calming interface design. The layout avoids unnecessary visual clutter and focuses on readability and ease of navigation. Simple interaction patterns are favored to reduce cognitive load and encourage consistent daily engagement.

To improve accessibility and flexibility, the system is designed to support both text-based and voice-based journal entries. This allows users to capture thoughts in different contexts and supports a wider range of user preferences.

Finally, the architecture is intentionally modular. Major system responsibilities such as user interaction, data storage, security, and external integrations are conceptually separated. This structure supports future enhancements, including analytics, personalization, and data synchronization, without requiring significant architectural changes.

3.2.0 User Interface

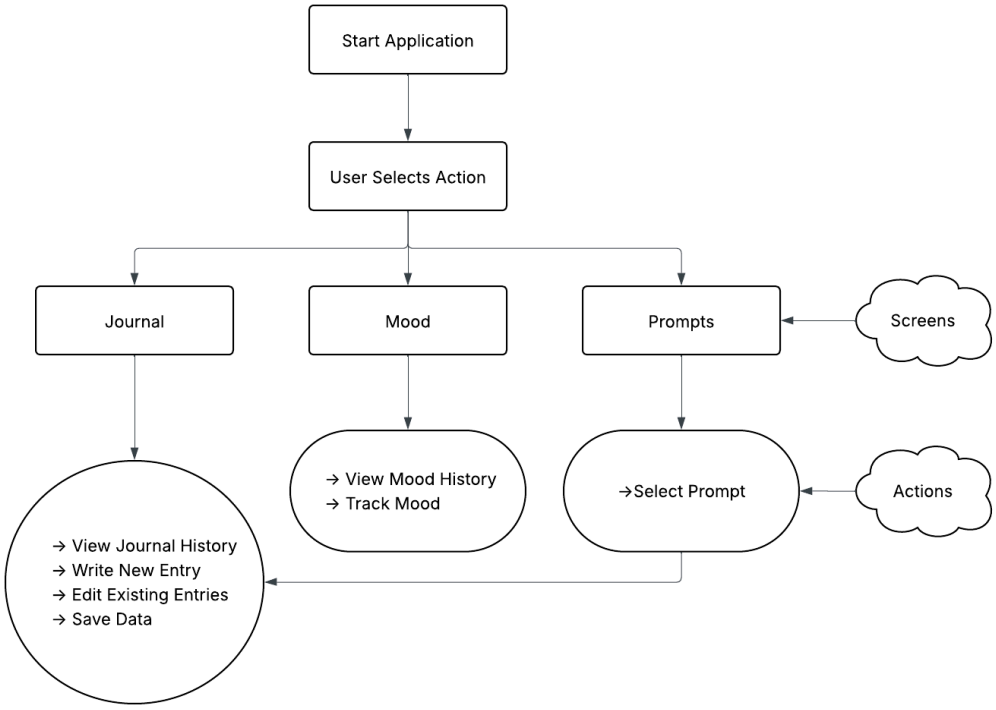
The user interface is designed with a strong emphasis on clarity, simplicity, and emotional comfort. The goal of the interface is to support reflection rather than distract from it. Navigation is designed to be intuitive, allowing users to move easily between journaling, mood tracking, and prompt exploration. Visual elements such as icons and color cues are used sparingly to provide feedback without overwhelming the user. Overall, the interface prioritizes ease of use, accessibility, and a calm visual tone suitable for a mental wellness application.

3.3 System Architecture

At a high level, the system architecture follows a layered design that separates user interaction, application logic, and data management. The presentation layer is responsible for displaying content and capturing user input. Beneath this layer, application logic coordinates journaling workflows, mood tracking, and feature interactions. Supporting services handle concerns such as data storage, security, and optional external integrations. Data is managed through well-defined interfaces to ensure consistency and maintainability. This decomposition improves clarity, reduces coupling between components, and supports future system growth.

3.4 Detailed System Design

This section provides a high-level description of the major software components that make up the proposed system. It explains how responsibilities are divided among different components and how these components are expected to interact. The focus is on conceptual design decisions rather than implementation details, as development has not yet begun. This section supports a clearer understanding of the system's internal structure and behavior.



3.4.0 Classification

The system is composed of several types of software components, each serving a distinct role. At the highest level, the system includes presentation components responsible for user interaction, application-level components that coordinate system behavior, and service components that manage data, security, and external functionality. Presentation components

include the screens through which users create journal entries, view past entries, track moods, and explore writing prompts. Application-level components manage navigation, workflow control, and interaction between user-facing screens and underlying services. Service components are responsible for handling persistent data storage, protecting sensitive information, managing audio input, and supporting optional intelligent features. This separation of components helps reduce coupling, improves clarity, and supports future system expansion.

3.4.1 Definition

Each major component in the system has a clearly defined responsibility. Presentation components are responsible for displaying information to the user and capturing user input in a clear and intuitive manner. These components do not manage data directly but instead communicate with other parts of the system through well-defined interfaces. Application-level components coordinate system behavior, ensuring that user actions result in appropriate system responses and that data flows correctly between components.

Service components handle specialized functionality that supports the rest of the system. Data management components are responsible for storing and retrieving journal entries, mood information, and saved prompts in a reliable and secure manner. Security-related components are responsible for protecting stored data through encryption and key management, ensuring user privacy. Audio-related components manage voice input and playback, allowing users to record and listen to voice-based journal entries. Optional intelligence-related components are designed to provide writing prompts and reflective feedback to enhance the journaling experience while remaining independent of core system functionality.

Together, these components form a cohesive system in which responsibilities are clearly divided, interactions are well defined, and future changes can be accommodated with minimal disruption.

3.4.2 Constraints

The system is designed to operate within the limitations of mobile devices, including restricted memory, processing power, and local storage. Components must respond to user actions in a timely manner to ensure a smooth user experience. The system assumes single-user interaction and intermittent use rather than continuous operation. Data-related components must preserve consistency when journal entries are created, modified, or removed. Certain operations require specific conditions to be met, such as user permission for audio input. Because many system operations are asynchronous, components must handle delays, invalid input, and failures gracefully while maintaining a consistent system state.

3.4.3 Resources

The system relies on several external resources, including local device storage, memory, processing capacity, and device hardware such as audio input and output. Optional features

may require network connectivity and remote processing resources. Resources must be acquired and released appropriately to avoid unnecessary consumption or conflicts. Since multiple operations may access shared data asynchronously, care must be taken to prevent inconsistent states. The system design minimizes the risk of deadlocks by avoiding complex resource dependencies and ensuring proper cleanup after operations.

3.4.4 Interface/Exports

Each system component provides a set of clearly defined services that other components can use without knowledge of internal implementation details. These services include operations for managing journal entries, storing mood data, handling security-related functionality, and supporting optional features. Interfaces specify the expected inputs and outputs of each service and define basic usage rules and assumptions. This approach promotes modularity, simplifies component interaction, and allows internal changes without affecting the overall system design.

3.5 Technology Platform and Tools

The application is designed as a cross-platform mobile system using modern mobile and web-based technologies. Development will be carried out using Expo and React Native, which allow a single shared codebase to support both Android and iOS platforms. The primary programming languages used will be JavaScript and TypeScript, supporting modular design and maintainability. The system leverages standard mobile device capabilities such as local storage, audio input, and secure data handling. External services may be used to support optional features, but the core application functionality is designed to remain independent of continuous network access.

4.0 Development

4.1 Implementation Detail Narrative

4.1.0 System Integration & Workflow Overview

The Mindful Micro-Journaling App is a mobile first system with a modular design, which handles user input, local data, and a potential AI service.

The summarized workflow proceeds as follows:

1. The user is presented with a prompt, which could be created with an AI service
2. The user interacts with the application through the user interface in order to enter a journal entry and record their mood
3. The user data is safely stored

4. Mood values are stored locally and aggregated for trend charts.

Integrated Process:

This app is composed of three primary layers:

- **UI layer (frontend)** - This was built using react native to handle user interaction.
- **App logic layer** - This layer processes user input, manages prompts, and controls workflows
- **Data layer** - This layer is responsible for data storage (local or cloud)

Layer Communication:

- **UI layer -** Captures user input (user journal entries, mood selection) and sends to the logic layer
- **Application logic layer -** Processes and validates user data then determines actions (save entry, generate prompt, update trends)
- **Data layer -** Stores entries and retrieves user data for analytics and visualization (graphs)

Operational Workflow:

1. Launching the app
    a. The user opens the application
    b. Saved user data is retrieved
2. Journaling Prompt
    a. A journaling prompt is displayed for the user
        i. In the current build, prompts use predefined or local fallbacks unless an API key is configured
3. User Entries
    a. User writes a journal entry
    b. User selects current mood
4. Storage
    a. Input is validated
    b. The user's entry is saved on local device storage only
5. Mood Analysis
    a. Data regarding the user's mood is collected.
    b. From this data, trends are calculated
6. Feedback
    a. User can view past journal entries
    b. Graphs regarding user data will be available for user to view
7. End Session
    a. The user is returned to the homescreen

Global Design Decisions:

1. Mobile priority architecture
    a. The app was designed primarily for mobile phone users.

2. Data Storage Strategy
    a. Local storage only for offline capability
3. Error Handling Strategy
   Input validation as well as try-catch are implemented on app to deal with:
    a. Invalid user input
    b. Network failure
    c. Data storage errors
4. Privacy Focused
    a. User data is securely stored and protected

4.1.1 User Interface Component

- Conceptual Overview:
    - The user interface component provides all screens for the user. The screens provide the user with journal prompts/entries, mood selection, and insights into user data. The UI design is meant to be simple and easy to use to match the theme of mindfulness and wellness.
- Development Process:
    - The app's UI was developed using React Native which also allows for cross platform compatibility.
    - Key steps:
        - Designing screen layout (journaling prompts, trends/insights, and journaling)
        - Implementing consistent UI patterns (buttons, inputs, cards)

4.1.2 Secure Local Data & Storage Layer

- Conceptual Overview:

This component handles how journal data is saved and retrieved. It includes entry storage, mood history, saved prompts, and daily prompt caching. It is also where privacy protection is applied through encryption before storing journal content.

- Development Process:

We created a dedicated storage service so all save/load/delete actions go through one place. This helped us avoid repeating logic across multiple screens. Journal entries are stored with useful fields like id, date, mood, prompt context, and optional voice reference. Mood history is managed separately, which keeps trend calculations simpler and more efficient.

For privacy, we added an encryption service that handles encryption key access and encrypts entry content before writing to local storage. During reads, entries are decrypted and sorted by

newest date first for display. This approach gave us better organization and a cleaner separation between UI code and technical data logic.

A practical issue we handled was reliability. Data or parsing errors can happen, so we used safe return values and graceful fallback behavior to avoid crashes. The goal was to make sure users can still interact with the app even if one data operation fails.

4.2 Database and External Connections

4.2.0 Connection Overview

At this stage, the app does not use a traditional cloud database (e.g. MySQL, PostgreSQL, MongoDB).

Primary persistence is local device storage (AsyncStorage), which supports offline use and keeps the scope manageable.

This choice matches our goals for performance, privacy, and reliability without depending on a hosted backend.

In the current build, AI-style features (prompts, affirmations, summaries) run using built-in fallback content defined in the app—no API key or external calls are required for normal use.

The codebase includes optional integration with an external AI provider for future development (e.g. after adding EXPO_PUBLIC_OPENAI_API_KEY in the dev environment). That integration is not required for core journaling and is not relied on in our current deployment.

4.2.1 Configuration Details

The application uses AsyncStorage for on-device persistence (journal entries, mood history, saved prompts, and daily prompt cache). expo-secure-store and expo-crypto support a local encryption layer so journal content is not stored in plain text.

The codebase includes optional integration with the OpenAI Chat Completions API via the environment variable EXPO_PUBLIC_OPENAI_API_KEY. This is not required for the app to run. In the current build, that key is not used, and prompt generation and related AI-style features rely on local fallback prompts defined in the app.

For future development, teams can enable live AI by adding EXPO_PUBLIC_OPENAI_API_KEY to the development environment (e.g. a local .env file that is not committed to the repository). No other configuration is required for core journaling, mood tracking, or local storage.

4.2.2 Integration Logic

UI screens call storage methods like save/get/delete.

Storage service encrypts journal content and writes it to local storage. On read, stored data is decrypted and transformed back into usable entry objects.

AI service sends structured requests to the OpenAI endpoint when a key is present. If API calls fail or no key is found, the service returns fallback prompts or affirmations so the app remains fully usable. In the current deployment, only fallback behavior is used unless a key is configured.

This layered integration made debugging easier and kept screen components focused on presentation rather than backend details.

4.3 Project Setup and Environment Configuration

4.3.0 Prerequisites**:**

To run the app from scratch, the following are needed:

1. Node.js (LTS recommended)
2. npm
3. Expo tooling (via project scripts)
4. Mobile emulator or Expo Go on a phone
5. Internet for dependency installation

Our stack includes React Native, Expo, TypeScript, React Navigation, AsyncStorage, Axios, and date formatting/chart libraries.

4.3.1 Installation Steps:

1. Install Node.js LTS.
2. Clone the repository and cd into the app folder (where package.json is).
3. Run npm install.
4. Run npm start.
5. Open the app:
    - Android: scan QR with Expo Go.
    - iOS: scan QR with Expo Go on a device
    - Web: press w in the Expo CLI, or run npm run web.

4.3.2 Verification:

We verify successful setup using the following checks:

- Expo starts without install/build errors.
- App launches on emulator/device/browser.
- Journal screen appears correctly.
- New entry can be created and saved.
- Saved data persists after reload.
- Mood values display correctly with entries.
- Random prompts are generated correctly based on the category chosen by the user
- User can create their own prompt and use it successfully
- User can create multiple journal entries per day; duplicate entries for the same prompt on the same day are blocked.

If all checks pass, the environment is correctly configured.

5.0 Test (plan and report)

5.1 Test Objectives

The objective of our testing is to verify that the Mindful Journaling App functions correctly, securely, and reliably. The testing will focus on the following:

- Users creating and storing journal entries
- Mood tracking properly recorded and associated with entries
- Data securely stored and persists across sessions
- Voice recording accurate and reliable
- App maintains performance and usability during all use cases

5.2 Scope of Testing

- In-scope: Features/modules we will test
    - User creation and storing of journal entries
    - Viewing previously saved journal entries
    - Local data storage and persistence across app restarts
    - Secure handling of stored journal data
    - Basic mood trend or mood history display
    - User interface functionality, navigation, and general usability
    - App performance during normal user interactions

- Out-of-scope: Features we will not test
  - Testing of third-party services beyond confirming they connect and respond correctly
  - Advanced penetration testing or formal security certification
  - Testing on every possible mobile device and operating system version
  - App store deployment, publishing, and marketplace approval testing

5.3 Test Cases

- List scenarios to test (functional and non-functional if applicable)
- Example format:
  - Test Case ID
  - Description
  - Expected Result

Test Case ID	Description	Expected Result
TC-01	Create a new journal entry	The user is able to: open the journal screen, type an entry, and submit it successfully
TC-02	Save a journal entry	Journal entry is saved successfully and is still available after restarting the app
TC-03	View past journal entries	Previously stored journal entries are displayed correctly in appropriate section
TC-04	Use audio recording feature	User is able to start and stop recording without any issues
TC-05	Save audio recording	The recorded audio is saved successfully and is able to be accessed later
TC-06	Track moods	The user's selected moods are recorded correctly and displayed in the mood tracking graph
TC-07	Auto generate daily affirmation	The app generates and displays a daily affirmation
TC-08	Verify stored data persists across sessions	Journal entries, mood data, and related content remain available after closing and reopening the app

TC-09	Check app usability during normal use	The app remains responsive, navigation works properly, and screens load without major delay
TC-10	Check voice/audio reliability	Audio feature works consistently during separated use without crashing

5.4 Test Procedures

Test Case ID	Procedures
TC-01	1. Launch application 2. Navigate to the journal entry screen 3. Enter text into the input field 4. Submit journal entry
TC-02	1. Submit journal entry 2. Navigate to journal logs screen 3. Exit app 4. Re-launch application 5. Navigate to journal logs screen
TC-03	1. Launch application 2. Navigate to journal logs screen
TC-04	1. Navigate to the journal entry screen 2. Press audio recording button 3. Speak a short sentence 4. Stop recording
TC-05	1. Navigate to journal logs screen 2. Attempt to play audio recording
TC-06	1. Navigate to journal entry screen 2. Submit mood 1-3 times 3. Navigate to mood screen 4. Confirm accuracy of graphs and data saved
TC-07	1. Navigate to journal screen 2. Ensure that daily affirmation is present 3. On a separate day, confirm that a different daily affirmation is present
TC-08	1. Launch application 2. Enter journal entry, mood data, and voice recording 3. Close application 4. Re-launch application 5. Navigate to journal logs screen 6. Ensure all data previously entered is present

TC-09	1. Ensure all expected data is present before moving onto next step 2. Navigate to journal entry screen 3. Enter a short journal entry, mood data, and voice recording 4. Navigate to mood screen 5. Select “week”, “month”, and “all” options on mood screen 6. Navigate to prompt screen 7. Generate a prompt 8. Submit a journal for prompt 9. Create a custom prompt 10. Submit journal for prompt 11. This process should take less than 2-3 minutes 12. Note any unexpected wait times or missing data.
TC-10	1. Navigate to journal entry screen 2. Enter multiple voice recordings of varying length 3. Navigate to journal logs screen 4. Play recordings previously submitted 5. Note any issues with creating recordings or saved recording

5.5 Test Environment

Our app was tested in Expo Go’s controlled development environment that simulates typical user conditions on a mobile device.

Hardware:

- Mobile Device (smartphone) used for real device testing using Expo Go
- Device microphone used for testing voice/audio recording features
- Personal computer used for development and testing

Software:

- Visual Studio Code used for development and debugging
- Node.js for running the development server
- Expo Go for testing the app on a physical device
- React Native for application development

Network Environment:

- Standard at home wi-fi used during app testing

Additional Tools:

- Local storage used to test data persistence

5.6 Test Data

The following test data was used to verify the functionality of the Mindful Journaling App. The data was designed to simulate realistic user input and ensure proper handling of different scenarios.

Sample Journal Entries:

Short entry:
"Focused more on goals"

Medium Entry:
"This month I have spent more time setting clear goals and meeting them. I try to organize my time better so that I can finish my daily tasks."

5.6.0 Mood Tracking Data

- Mood selections such as:
  - Happy
  - Neutral
  - Sad
- Multiple entries across different days to simulate weekly mood tracking
- Repeated moods to test consistency in trend calculations

5.7 Software Test Report (STR)

Requirement	Pass	Fail	Severity
Create journal entry	Pass	N/A	Critical
Save journal entry	Pass	N/A	Critical
View past entries	Pass	N/A	Critical
Use audio recording	Pass	N/A	Moderate
Save audio recording	Pass	N/A	Moderate
Track moods (weekly)	Pass	N/A	Critical
Auto generate daily affirmation	Pass	N/A	Moderate

Data persistence across sessions	Pass	N/A	Critical
App usability and performance	Pass	N/A	Critical
Voice/audio reliability	Pass	N/A	Critical

6.0 Summary

The Green Mindful Micro-Journaling App brings together daily reflective prompts, mood tracking, and mindfulness techniques to encourage consistent self-reflection without overwhelming the user. By emphasizing brief entries that can be completed in minutes, the application lowers the barrier to maintain a journaling habit. By combining simplicity, privacy, and technology, meaningful mental wellness support can be delivered effectively through our mobile Green Mindful Micro-Journaling app.